11f — Twelve-Factor Apps

Key Concepts

- **Twelve-Factor App** — methodology of 12 principles for building scalable, cloud-native applications
- Developed by **Heroku engineers** based on patterns across thousands of production apps
- Goal: apps that are portable, scalable, and deployable across environments without code changes

The 12 Factors

#	Factor	Core Rule
1	**Codebase**	One repo, many deploys (dev/staging/prod)
2	**Dependencies**	Declare all dependencies explicitly (NuGet, npm) — no system-installed assumptions
3	**Config**	Store config in environment variables — never hardcode secrets or connection strings
4	**Backing Services**	Treat DB, cache, queue as interchangeable attached resources — swap via config only
5	**Build, Release, Run**	Strictly separate build → release → run stages
6	**Processes**	Run as stateless processes — no in-memory sessions, no local file storage
7	**Port Binding**	App is self-contained and exposes itself on a port (e.g. Kestrel in .NET)
8	**Concurrency**	Scale out horizontally (more instances) not vertically (bigger server)
9	**Disposability**	Fast startup, graceful shutdown — finish current requests, stop accepting new ones
10	**Dev/Prod Parity**	Dev, staging, prod as identical as possible — eliminate "works on my machine"
11	**Logs**	Write to stdout — environment collects and stores, app never manages log files
12	**Admin Processes**	Run admin tasks (migrations, scripts) as separate one-off commands, not on app startup

Three Most Important for Interviews

Factor	Why It Matters
Config (3)	Security — hardcoded secrets is a vulnerability; same build deploys to any environment
Processes (6)	Statelessness enables horizontal scaling — add instances without coordination

Factor	Why It Matters
Dev/Prod Parity (10)	Eliminates environment-specific bugs — use Docker to match environments exactly

Build, Release, Run — How It Works

```
Build   → compile code, pull dependencies
Release → combine build + environment config
Run     → execute the release
```

- Never change code at runtime
- CI/CD pipeline enforces this separation

Stateless Processes — Scaling Impact

```
Scale UP (vertical)    → bigger server       ❌ not twelve-factor
Scale OUT (horizontal) → more instances      ☑ twelve-factor
```

- Any instance can handle any request
- No sticky sessions, no in-memory state

.NET Relevance

Factor	.NET Example
Config	Azure Key Vault + environment variables, not appsettings.json for secrets
Port Binding	Kestrel — app self-hosts, binds to port
Admin Processes	`dotnet ef database update` as separate command, not on app startup
Logs	Write to stdout → Azure Monitor / Application Insights collects

Industry Examples

- **Heroku** — platform built around these principles; apps that follow them deploy seamlessly
- **Docker** — Factor 10 (Dev/Prod Parity) — containers make local env match prod exactly
- **Kubernetes** — Factor 9 (Disposability) — fast startup/graceful shutdown required for pod scaling

Interview Questions

Q: What is the Twelve-Factor App methodology? A: A set of twelve principles by Heroku engineers for building scalable, cloud-native apps that are portable and deployable across environments without code changes.

Q: Why should config be stored in environment variables? A: Same build artifact deploys to any environment without changes, and secrets are never hardcoded in the codebase — reducing security risk.

Q: What does stateless processes mean and why does it matter? A: Each process holds no in-memory state between requests — all state lives in a backing service. Enables horizontal scaling — spin up any number of identical instances.

Q: What is dev/prod parity and why does it matter? A: Keeping dev, staging, and prod environments as identical as possible — same database engine, same config structure — to eliminate environment-specific bugs.